Grocer-E - Ethan Fowler and Koen Tanner

Standup Slot	Mondays from 12-12:15
User Representative	Trevor Basham tbasham2@murraystate.edu

Project Overview

At a high level, Grocer-E is an online order management service designed for grocery stores. Employees will use it to fulfill orders placed by customers, taking items off the sales floor and organizing them to be sent out later. Customers will use this app to place their orders and schedule them for later pickup. Grocer-E will also utilize an AI pathfinding algorithm and multiple user statistics, such as pick rate, to inform the efficient management of the store. Grocer-E seeks to redefine the online ordering landscape by enabling lower customer wait times and higher accuracy picking compared to competitors, such as Walmart's Global Integrated Fulfillment service.

EPIC 1: As a manager, I want to be able to manage the layout of store items so that shoppers and pickers can be as efficient as possible.

- As a manager, I want to know information about the items on the shelves so that I can determine if they're organized efficiently.
    - **Frontend Tasks**
    - ☐ Create an inventory management screen that fetches the list of items in the store
    - ☐ By default, this list shall list the items in order by aisle and section
    - ☐ Add a filtering and search bar UI; filters should include aisle/section, items with no location only, alphabetical order, reverse alphabetical order, reverse On Hand order, item category
    - ☐ Add a filter by aisle number
    - ☐ Refresh the UI each time the search conditions are modified
    - **Backend Tasks**
    - ☐ CRUD READ pipeline to fetch the list of items as a JSON object
    - ☐ Establish different ORDER BY and GROUP BY SQL functions to be called when different filtering options are selected
    - **Testing Tasks**
    - ☐ Add tests that verify each table filter displays the contents correctly
    - ☐ Add a test to verify that only items with a blank Locations field appear in the list when the No Location filter is applied, and a message reading "All caught up! No items need locations added" appears in place of a table when no items have a blank Locations field
    - **Documentation Tasks**
    - ☐ Add to documentation an explanation of what is considered an efficient store pattern
    - ☐ Add to documentation common use cases for the different filters:
        - ☐ By aisle/section: Default; use to check the aisles one by one

        - ☐ By item category: Identifies items from one department that may be scanned into the area of a different wrong department
        - ☐ No locations only: Identifies items with no known location that may be difficult to locate in pick walks and when customers shop
        - ☐ Alphabetical order and reverse alphabetical order: Locate a specific item by name if you can't remember the exact name for the search bar
        - ☐ Reverse On Hand order: Identifies items out of stock or close to out of stock
- As a manager, I want to be able to change the locations of items in the store so that efficiency is maintained as different items come and go.
    - **Frontend Tasks**
    - ☐ Within the UI displaying individual item details, add an Edit feature to the Location section
    - ☐ This Edit button should display a dialog allowing the user to either add a new location or edit/remove an existing location
    - ☐ Dialog should warn the user if they try to delete the last remaining location for an item
    - ☐ Add an Edit capability to the On Hand figures for each item
    - **Backend Tasks**
    - ☐ Add CREATE/UPDATE/DELETE pipelines so that form data from location modifications go to those item rows in the database
    - ☐ Add UPDATE pipeline to handle On Hand updates as inventory changes from shopping, damages, truck shipments, etc.
    - **Testing Tasks**
    - ☐ Location validity checks: this app plans to use a Zone letter - Aisle number - Section number - Modular number pattern
        - ☐ Zone field must be a letter and cannot be blank
        - ☐ Aisle number must be an integer and cannot be blank
        - ☐ Section number must be an integer, if provided
        - ☐ Modular number must be an integer, if provided
        - ☐ This means a location can be as broad as a zone and aisle number to account for stores that don't employ a more specific system
    - ☐ Verify the On Hand figure for any given item is non-negative and an integer
    - **Documentation Tasks**
    - ☐ Document CRUD pipelines for this set of features
- As a manager, I want to be able to devise a path from the item locations so that pickers can be as efficient as possible when fulfilling orders and so that customers can find items in logical locations.
    - **Frontend Tasks**
    - ☐ Add screen to add, move, or remove potential locations
    - ☐ Add screen to customize path from these locations
    - ☐ Use dropdowns to change the order in which zones are visited

- ☐ When picking, item displayed should match the next location appearing in the order list
- ☐ Generate a map representation of the current layout and the path; when a dropdown changes the order of items in the path, the new path should be shown before the user can save it
- **Backend Tasks**
- ☐ Generate a linked list based on available locations
- ☐ A node in the list points to its neighbor to determine which item is visited next
- ☐ Include a check such that if a location in the store is not included in the order list, it is skipped until a location is reached that appears in the order list
- ☐ Incorporate an AI pathfinding algorithm via API to generate the best available path or suggest weak points in existing paths
- **Testing Tasks**
- ☐ Verify the location and item appearing on the picking screen is actually the next item in order
- ☐ Verify locations not appearing in the order list do not also appear while picking
- ☐ Verify items with no known location appear during picking after all other items with locations

EPIC 2: As a picker, I want to be able to pick items off the shelves so that I can fulfill online orders as quickly as possible.

- As a picker, I want to be able to pick the item listed on my screen so that I can complete a pick walk.
  - **Frontend Tasks**
  - ☐ Create picking UI containing the following information:
    - ☐ Item name
    - ☐ Item image
    - ☐ Quantity to be picked
    - ☐ Order number requesting the item (if picking multiple orders at once)
    - ☐ UPC
    - ☐ Price
    - ☐ On Hand figure
    - ☐ Customer instructions, if any
    - ☐ Items picked and items remaining in current walk
    - ☐ Current pick rate
    - ☐ Open Camera button to prompt the camera
    - ☐ Item Not Found button
    - ☐ Exit Walk button
    - ☐ Pause button
  - ☐ After the correct item is scanned, advance screen to show next item in path
  - ☐ After an incorrect item is scanned, display an error dialog
  - ☐ After pressing Item Not Found:

        - ☐ If the customer specified a substitute item AND it is the next closest item, display it for picking
        - ☐ If the customer specified a substitute item AND it is NOT the next closest item, that item is added to the pick list in path order, and the next closest item in the path is displayed for picking instead
        - ☐ If the customer did not specify a substitute, advance to next item
    - ☐ After scanning the correct item and the Quantity box is >1, show another dialog telling the picker to enter the number of items actually picked
    - **Backend Tasks**
    - ☐ App must be able to access the phone's camera to scan UPC barcodes
    - ☐ App must call to a UPC lookup API to process the UPC into data to be checked against the database
    - **Database Tasks**
    - ☐ Each item removed from the salesfloor during a pick walk must reduce the On Hand for that item by 1; multiples of the same item should remove the number in the Quantity box in the UI
    - ☐ Each UPC scan should prompt a database search for a matching UPC corresponding to the intended item
    - **Testing Tasks**
    - ☐ If a QR code is accidentally scanned instead, a dialog should be displayed saying that this code isn't valid
    - ☐ If a UPC scanned matches a different item than the one intended, or if the UPC search returns -1, a dialog should appear saying this is the wrong UPC
    - ☐ If the correct UPC is scanned, the pick walk should proceed
    - **Documentation Tasks**
    - ☐ Include general usage instructions in the README for edge cases, such as instances where a picker should alert management about errors:
        - ☐ Item found in a location other than the listed ones
        - ☐ Item found with On Hand of 0
        - ☐ Item not found with an On Hand far greater than 0
        - ☐ Items not suitable for picking or damaged
    - **Hardware Considerations**
    - ☐ Various grocery items will be required
    - ☐ Materials to designate locations will be required
    - ☐ Physical cart or other container to hold picked items will be required
- As a picker, I want to be able to pick oversized or different-temperature items separately so that orders can be fulfilled faster while maintaining employee and customer safety.
    - ☐ Available walks separated in UI screen
    - ☐ Selecting a walk type shows the different orders ready to be shopped for that category
    - ☐ Selecting an order starts a walk

- ☐ Items in orders should be categorized into Ambient/Chilled/Frozen/Hot and copied into tables based on those types
- ☐ Items separated into walks based on temperature and size restrictions
- ☐ Ambient and Chilled items should not mix in the same walk, Chilled and Frozen should not mix, etc.
- ☐ Extremely large items should appear in oversized walks only and not in the others

- As a picker, I want to know how fast I am picking on a given day so that I know where to improve my own efficiency.
  - **Frontend Tasks**
  - ☐ Include a UI screen showing the picker's stats as pulled from their database row
  - ☐ Stats should include:
    - ☐ Pick rate for that day
    - ☐ Lifetime pick rate
    - ☐ Items picked that day
    - ☐ Lifetime items picked
    - ☐ First-time pick rate that day
    - ☐ Pre-substitution that day
    - ☐ Post-substitution that day
    - ☐ Lifetime orders fulfilled
    - ☐ Store pick rate that day
    - ☐ Store orders fulfilled that day
    - ☐ Store total orders that day
    - ☐ Store items picked that day
    - ☐ Store items left to pick that day
  - ☐ Include a UI showing a leaderboard containing pickers' stats for that day and assigning rank based on pick rate
  - **Backend Tasks**
  - ☐ READ/UPDATE pipelines to show and change the stats
  - ☐ CRUD pipelines for accounts
  - ☐ Reset daily rates at midnight
  - **Database Tasks**
  - ☐ Data for a picker should be fetched as a JSON object to be rendered in the UI
  - ☐ Table of employees
    - ☐ Employee information should contain: name, username, password (encrypted), email
    - ☐ Statistical columns for the stats listed above
  - **Testing Tasks**
  - ☐ Statistical validation for stats above
  - ☐ Minimum character limits for password security
  - ☐ Form validation for logging in

- **Documentation Tasks**
  - ☐ Document statistic definitions and what affects them

EPIC 3: As a backroom employee, I want to be able to manage online orders and their contents so that they can be fulfilled accurately and efficiently.

- As a backroom employee, I want to know which customers have arrived to pick up their orders so that I can fulfill them first.
  - ☐ UI displays order numbers, arrival status
  - ☐ Fetch order info as a JSON from the order table
  - ☐ Orders disappear from database after fulfilled to keep clutter down
- As a backroom employee, I want to be able to organize items from various pick walks so that the correct items are with the correct orders.
  - ☐ Staging screen UI displaying totes or carts needing to be staged in locations
  - ☐ As items are picked, ambient items not already in the ambient cart for the order need to be combined with that cart
  - ☐ API for UPC processing required
  - ☐ UPCs must be generated for totes or carts to organize them
  - ☐ Additional UPCs must be created for tote locations
  - ☐ When staging, the UPC of the cart/tote is scanned, followed by the location
  - ☐ Unit testing: if a UPC scanned matches an item and not a tote when staging, an error message should be displayed
  - ☐ If a second location UPC is scanned when staging the same order, a warning should be displayed asking if the action is really intended
  - ☐ Once a customer arrives, a prompt should be displayed to combine the cold items with the ambient items in the order's cart
- As a backroom employee, I want to verify that the correct items are going to the correct customers so that I can maintain accuracy and customer satisfaction.
  - **Frontend Tasks**
  - ☐ UI should warn employee of substitutions in an order
  - ☐ UI should inform employee of removed/cancelled items so that they don't go to the customer
  - ☐ UI should provide the ability for employees to check the order's list one last time before confirming the dispense
  - **Backend Tasks**
  - ☐ Fetch shopper list as a JSON
  - **Testing Tasks**
  - ☐ Verify items displayed belong to the customer

EPIC 4: As a shopper, I want to be able to shop in-store and online so that I can get groceries and fulfill life needs, even when unable to travel.

- As a shopper, I want to shop the items in the store online so that I can save time otherwise spent navigating physical aisles.
  - **Frontend Tasks**
  - ☐ On shopper's app, display items for sale in the store

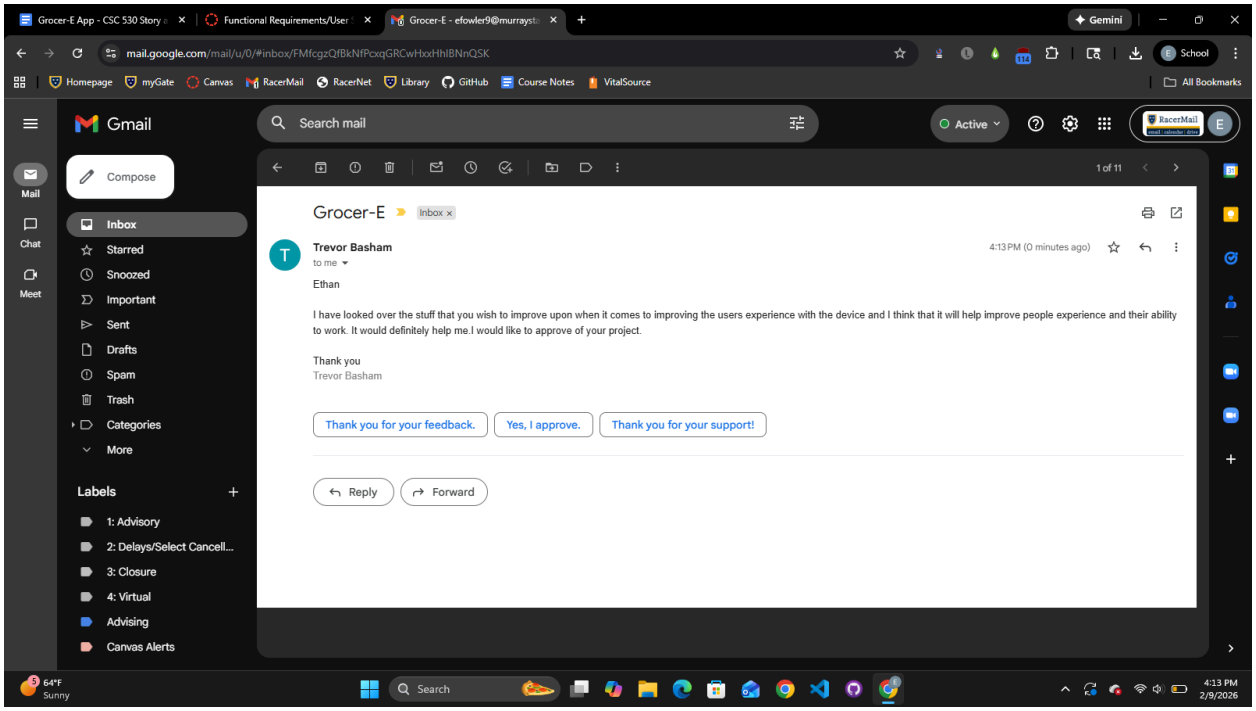
- ☐ An item should display the following information on the home page as a card:
    - ☐ Picture
    - ☐ Name
    - ☐ Unit price
    - ☐ On Hand count
    - ☐ Add to Cart button
    - ☐ Quantity adjustment button (for bulk buys)
- ☐ If the user selects an item's card, but not the Add to Cart button, another screen should populate with additional information
    - ☐ Info above, plus:
    - ☐ Description
    - ☐ Department
    - ☐ In-store location(s)
- ☐ Order scheduling
    - ☐ A button at the top of the page for the user's cart
        - ☐ This button should have a red dot with the number of items currently in the cart over the icon
    - ☐ When the Cart button is clicked, the list of items in the cart should be displayed to the user
    - ☐ For each item, the user should have a chance to adjust quantities, remove items, or nominate substitutions
    - ☐ If any item is priced by weight, the user should receive a warning saying that the final total will vary based on the weight of the items received
    - ☐ A calendar and timeslot UI should appear to let the customer select an available time for pickup
        - ☐ Timeslots available for selection must meet the following criteria:
            - ☐ 20 or less orders already placed for that slot
            - ☐ Only slots greater than 3 hours after the current time are available for new orders; for example, if it's 12:30 PM, the earliest slot would be 4:00 PM
            - ☐ Orders cannot be scheduled more than 7 days in advance
- ☐ Prepayment
    - ☐ For security reasons, payment methods shall not be saved for reuse
    - ☐ Fields for cardholder name, card number, expiration date, and CVV
    - ☐ Payment screen should show an estimated total
        - ☐ If the user allowed substitutions, warn that the total can change if the original items are unavailable
        - ☐ If the user has items priced by weight, warn that the total can change based on the weight of those items
- **Backend Tasks**
- ☐ Payment processing API will be required

- ☐ CRUD pipelines for cart management, order placement, and item population and updating
- ☐ Ideally, On Hand stats should update in real time without page refreshment
- **Database Tasks**
- ☐ Customer clients should have their own carts
- ☐ Each timeslot should have a table of scheduled orders, with a limit of 20
- ☐ When an order is placed, the contents of the user's cart should be "cut" from their client's cart table and placed in an order details table, itself a row in the timeslot table
- ☐ Each timeslot table should have at least the following columns:
    - ☐ Order number (primary key, unique, not null)
    - ☐ Name (varchar, not null)
    - ☐ Items (foreign key to their list?)
- **Testing Tasks**
- ☐ Verify that timeslots can't get more than 20 orders
- ☐ Verify that new orders can't be scheduled less than 3 hours after the current time
- ☐ Verify that orders can't be placed more than 7 days beyond the current date
- ☐ Verify that cards used for payments are legitimate
- ☐ Verify that item cards and information pages render the correct information
- **Documentation Tasks**
- ☐ Document CRUD paths between the tables needed for this story
- ☐ Document API usage and payment processing and data storage policy

- As a shopper, I want to know the current status of my online orders so that I have peace of mind that my groceries are being handled as accurately as possible.
    - **Frontend Tasks**
    - ☐ Shopper UI should display a header at top of page when an order is placed but has not been picked up yet
    - ☐ Header should display fulfillment status
        - ☐ Scheduled
        - ☐ Picking in progress
        - ☐ Pick complete
        - ☐ Staged
        - ☐ Ready for pickup
        - ☐ Substitutions to review
        - ☐ Canceled
        - ☐ Delayed
        - ☐ Complete
    - ☐ Include a button in shopper UI to call the store
    - ☐ Include a button in employee's order list screen to call the customer
    - **Backend Tasks**
    - ☐ Both employee and shopper apps need calling permissions enabled

    - ☐ Shopper header should fetch their order's details from database as a JSON object
    - ☐ Employee order list screen should be built from the full fetched list
    - **Database Tasks**
    - ☐ Include a column in the Daily Orders table indicating order status
    - **Testing Tasks**
    - ☐ Header should not appear on shopper's screen if there are no pending orders for that customer
    - ☐ UI should reflect correct status
- As a shopper, I want to know which items are available and in stock at a given store so that I can place orders for those items without worrying about them being unavailable.
    - **Frontend Tasks**
    - ☐ Shopper UI displays all items in the store
    - ☐ Shopper UI grays-out items listed with On Hand of 0
    - ☐ Add to Cart button is disabled or missing on items out of stock
    - ☐ Add to Cart button enabled on items in stock
    - ☐ Quantity picker UI upper bound must match the current On Hand total for a given item
    - **Backend Tasks**
    - ☐ CRUD READ pipeline to fetch item data from database and display it in the shopper UI; this pipeline should be called every time the page is refreshed at minimum
    - ☐ CRUD READ/UPDATE/DELETE pipelines to add and remove items from the shopper's cart; the DELETE operation should be automatically called on items in the shopper's cart if the On Hand falls to 0
    - **Database Tasks**
    - ☐ Ensure all tables of items contain an On Hand integer column
    - ☐ Each user needs their own backend table or stack to act as their cart
    - **Testing Tasks**
    - ☐ Create unit test that confirms that a shopper cannot add an item that is out of stock to their cart
    - ☐ Create unit test checking that a shopper cannot add more of an item to their cart than the current On Hand figure for that item
    - ☐ Create unit test that ensures items in the cart are removed if their On Hand drops to 0 when the page refreshes
    - **Documentation Tasks**
    - ☐ Keep CRUD information up to date, as well as any other needed information for backend, frontend, and anything in between

Proof of User Buy-in:
"Ethan

I have looked over the stuff that you wish to improve upon when it comes to improving the users experience with the device and I think that it will help improve people experience and their ability to work. It would definitely help me.I would like to approve of your project.

Thank you
Trevor Basham”